UFX_AUTO_2

Database Fillout Work Order: Phases 6–10

VSEPR-SIM · v5 beta9 target

`Prerequisite: tiny/medium continual run stable (Phases 2-5 complete)`

Phase 2 fills the skeleton. Phases 6–10 give it something to say.

Contents

Purpose and Context

Phase 2 generated the atomistic identity and force-field spine. It proved the generation loop, the validation path, the web cache, and the continual harness. After Phase 2, the database holds approximately 100 unique `material_key` records before the axis space saturates, because only five axes vary: *element*, *oxidation_state*, *coordination*, *geometry*, and *phase*.

> Phases 6–10 are not a rewrite. They are additive. Each phase extends the existing `UFXMaterialRecord`, expands the generation axis space, adds one or more generator classes, and wires one or more new CLI sub-commands into the existing `ufx auto2` dispatch chain. The harness loop does not change.

Every phase follows the same template as Phase 2:

1. Extend `AxisConfig` with new sampling axes
2. Implement a `*Generator` class that fills the corresponding record block
3. Extend `LocalSanityChecker` with tier-appropriate checks
4. Insert block data into `property_values` with correct `block_tier`/`block_name`
5. Add a CLI sub-command: `ufx auto2 fill-<tier>`
6. Wire into the continual harness cycle after `randomfill`

Axis space growth by phase

Phase	Block	New axes	Approx. unique key growth
2	Identity + Force-field	5	∼100
6	Molecular descriptors	+4	×10
7	Thermo + EOS	+6	×20
8	Crystal / solid-state	+5	×30
9	Transport + Mechanics	+6	×25
10	Meta-score active scheduler	n/a	coverage-driven steering

> Phase 10 does not add new record axes. It steers the sampler toward under-covered regions of the combined axis space defined by Phases 2–9. Without it, the continual loop will over-generate common elements at standard conditions and under-represent actinides, high-pressure phases, and exotic geometries.

Shared Infrastructure (Review Before Phase 6)

These components are already built and must be understood before extending them.

AxisConfig

`include/ufx_auto2/axis_config.hpp` defines the sampling envelope. `AxisConfig::default_phase2()` populates five axis vectors. Phases 6–9 each call a new static builder, `AxisConfig::extend_phase`*N*`()`, that adds axes to an existing config rather than replacing it.

AxisSample

include/ufx_auto2/axis_config.hpp also defines AxisSample. New axes are added as fields here. Use std::optional<> for axes that are not populated in all phases.

```
struct AxisSample {
        // Phase 2 (mandatory)
        std::string element_family;
        std::string element;
        int         oxidation_state;
        int         coordination;
        std::string geometry;
        std::string phase;
        double      temperature_K;
        double      pressure_atm;
        std::string target_property;
        int         sample_index;
        uint64_t    seed_used;

        // Phase 6 -- molecular
        std::optional<std::string> canonical_smiles;
        std::optional<int>         heavy_atom_count;
        std::optional<double>      molecular_weight;

        // Phase 7 -- thermo
        std::optional<double> delta_Hf_kJ_mol;
        std::optional<double> cp_298_J_mol_K;
        std::optional<std::string> eos_model;

        // Phase 8 -- crystal
        std::optional<std::string> space_group;
        std::optional<double>      lattice_a_angstrom;
        std::optional<std::string> structure_source;

        // Phase 9 -- transport/mechanics
        std::optional<double>      strain_rate;
        std::optional<std::string> test_environment;
};
```

property_values insert pattern

Every new block value is inserted via ufx_insert_property_value() using the correct block_tier and block_name from the table below. State columns (temperature_K, pressure_Pa, phase) are always populated. Do not leave them at default without intent.

Tier	block_name	Phase	Example property_name
0	identity	2	coordination_number, geometry_tag
1	force_field	2	r1, D1, theta0
2	molecular	6	molecular_weight, inchikey
3	thermo	7	delta_Hf_kJ_mol, cp_298
3	eos	7	density_kg_m3, viscosity_Pa_s
4	crystal	8	space_group, density_g_cm3
5	mechanical	9	E_eff_GPa, yield_MPa
5	transport	9	k_eff_W_mK, diffusivity_m2_s
9	meta	10	weirdness, coverage, composite_score

Phase 6 — Molecular Descriptor Importer (Tier 2)

Goal: attach `MolecularDescriptorBlock` data to existing generated records, validate via PubChem PUG REST, and expand the axis space with molecular-formula and structural dimensions.

Why Phase 6 first

Molecular descriptors are cheap to generate (formula arithmetic) and cheap to validate (one PubChem CID lookup). They act as the first meaningful external-reference check after the Phase 5 web cache is warm. They also unlock the Tier 2 sanity filter: a generated `CH3Br` entry with the wrong molecular weight should fail before thermo data is ever computed.

New axis dimensions

- **canonical_smiles** — sampled from a SMILES vocabulary per element family
- **heavy_atom_count** — integer 1–20
- **hbond_donor_count** — integer 0–6
- **hbond_acceptor_count** — integer 0–8

SMILES are not random strings. They are drawn from a curated per-element table that encodes known coordination/oxidation combinations. This table lives in `include/ufx_auto2/smiles_vocab.hpp`.

Generator class: MolecularDescriptorGenerator

File: `src/ufx_auto2/molecular_descriptor_generator.cpp`

```
class MolecularDescriptorGenerator {
public:
        MolecularDescriptorGenerator();

        // Fill rec.molecular from axis sample + periodic table data.
        // Does NOT call the network.  All values are deterministic.
        void fill(UFXMaterialRecord& rec, const AxisSample& s) const;

        // Compute molecular weight from elements + stoichiometry.
        static double molecular_weight_from_formula(const std::string&
            formula);

        // Build a canonical InChIKey approximation from element + oxidation.
        static std::string inchikey_stub(const std::string& element,
                                                                    int
                                                                        ox_state
                                                                        ,
                                                                    int
                                                                        coordination
                                                                        )
                                                                        ;


private:
        // Periodic table atomic weights (local; no external lookup).
        static double atomic_weight_(const std::string& sym) noexcept;
};
```

The generator fills molecular_formula, molecular_weight, exact_mass, heavy_atom_count, and a stub inchikey. It does not attempt SMILES generation from scratch — that is a Phase 6b concern. For Phase 6, SMILES is either sampled from the vocabulary table or left empty.

LocalSanityChecker extension

Add the following Tier 2 checks to LocalSanityChecker::check():

- molecular.molecular_weight > 0 — must be positive
- molecular.heavy_atom_count >= 1 — at least one heavy atom
- molecular weight is consistent with formula (within 5% tolerance)
- inchikey is non-empty if identity block is fully populated

Failure mode tag: molecular_descriptor_invalid.

property_values rows written by fill-molecular

property_name	block_tier	units	source
molecular_weight	2	g/mol	periodic_table_arithmetic
exact_mass	2	g/mol	periodic_table_arithmetic
heavy_atom_count	2	count	formula_parse
hbond_donor_count	2	count	smiles_count
hbond_acceptor_count	2	count	smiles_count
inchikey	2	string	inchi_stub

Web cross-check (validate-web extension)

The Phase 5 PubChem fetcher already retrieves molecular_weight and inchikey. Phase 6 adds a comparison step inside WebValidator::validate_batch():

1. Lookup PubChem CID by formula or element symbol
2. Compare MolecularWeight response field vs. stored property_values row (block=molecular, property=molecular_weight)
3. If relative error > 2%: write validation_records row with passed=0, note molecular_weight_mismatch
4. If CID not found: write passed=1, note web_missing, confidence capped 0.85

CLI sub-command

```
vsepr-ufx ufx auto2 fill-molecular --batch 500 --db ufx_auto2.sqlite
vsepr-ufx ufx auto2 fill-molecular --batch 500 --db ufx_auto2.sqlite --
    verbose
```

Processes records where block_name='molecular' rows are absent in property_values. Writes molecular descriptor rows, then queues for web cross-check on the next validate-web pass.

Continual harness integration

Insert after randomfill, before validate:

```
# Cycle order with Phase 6:
randomfill  --count $BatchSize
fill-molecular --batch $BatchSize
validate    --batch $ValidateBatch
validate-web --batch $WebBatch
promote     --min-score $MinScore
audit
```

Audit success criteria after Phase 6

- `Total property values` nonzero and growing each cycle
- `Missing provenance = 0` (fill-molecular writes provenance rows)
- At least one `molecular_weight_mismatch` in rejected reasons (confirms the cross-check fires)
- Web validation: `web missing` acceptable, `web pass` nonzero

Phase 7 — Thermo + EOS Blocks (Tier 3)

Goal: populate `ThermoBlock` and `EOSBlock` for generated records using NIST WebBook as the primary external reference, periodic trend heuristics as the generation fallback, and T/P-aware property curves stored in `property_values`.

Axis expansion

- **delta_Hf_kJ_mol** — sampled from heuristic periodic trend ranges
- **cp_298_J_mol_K** — seed value; full curve generated from Shomate heuristic
- **eos_model** — one of: `ideal`, `vdW`, `PR`, `tabular`
- **temperature_range** — (T_min, T_max) bracket for the property curve
- **phase_transition_flag** — boolean; indicates whether the record spans a phase boundary in the sample range
- **vapor_pressure_seed** — log10 seed value at 298 K

Generator class: ThermoGenerator

File: `src/ufx_auto2/thermo_generator.cpp`

```
class ThermoGenerator {
public:
        ThermoGenerator();

        // Populate rec.thermo using:
        //   1. Element-family heuristic ranges for Hf, Cp, Tb, Tm
        //   2. Shomate polynomial estimate (6 coefficients from group/period
            )
        //   3. EOS parameter estimates from critical property correlations
        void fill(UFXMaterialRecord& rec, const AxisSample& s) const;

        // Build a Cp(T) curve from Shomate heuristic in J/(mol K).
        static PropertyCurve estimate_cp_shomate(int Z,
```

```
        // Estimate critical properties via Lee-Kesler correlations.
        static void estimate_critical_properties(UFXMaterialRecord& rec)
            noexcept;

private:
        // Group/period offsets for Cp seed values.
        static double cp_seed_(int period, const std::string& family)
            noexcept;
        static double Hf_seed_(int Z, int ox_state)
            noexcept;
};
```

Shomate heuristic outline

The Shomate polynomial is $C_p(T) = A + BT + CT^2 + DT^3 + E/T^2$. For Phase 7, coefficients A–E are seeded from a lookup by (period, element family):

- A_0 from monatomic Dulong-Petit limit: $3R \approx 24.9$ J/(mol K)
- B, C from polynomial regression offsets per period (stored in a 7-row table)
- D, E are zero for Phase 7 (first-order heuristic only)

This is explicitly labelled `method_tag = "shomate_heuristic_phase7"` and `confidence = 0.35` in the provenance row. No false authority.

EOS model selection rules

Condition	EOS assigned
phase = gas, $T > 2T_c$	ideal
phase = gas, $T \leq 2T_c$	Peng-Robinson
phase = liquid	Peng-Robinson
phase = solid	tabular (density only)
phase = molten_salt	Peng-Robinson + salt correction flag

NIST WebBook fetcher extension

The Phase 5 `NISTFetcher` is extended with two new query modes:

```
// Fetch Cp(T) series from NIST thermochemical tables.
WebResponse fetch_cp_curve(const std::string& formula,
                                                    double T_min_K, double
                                                        T_max_K);

// Fetch phase transition temperatures (Tm, Tb).
WebResponse fetch_phase_transitions(const std::string& formula);
```

Both go through `CachingFetcher` — no raw network calls outside the decorator. Cache keys include formula, T_min, T_max rounded to 50 K bands.

LocalSanityChecker extension (Tier 3)

- `thermo.melting_point_K > 0` if phase = solid
- `thermo.boiling_point_K > thermo.melting_point_K` if both present
- `cp_T` curve has at least 2 points and is monotonically valid (no negative Cp)
- `delta_Hf_298` is in physically plausible range for the element family (actinides: −1500 to +200 kJ/mol; noble gas: always near 0)
- `eos.eos_model` is non-empty

Failure tag: `thermo_block_invalid`.

property_values rows written by fill-thermo

Thermo properties are stored as multiple rows, one per T/P point on the curve. The `temperature_K` column is the state variable; `pressure_Pa` defaults to 101325.0 for standard-state values.

property_name	units	note
delta_Hf_kJ_mol	kJ/mol	single row, T=298.15
delta_Gf_kJ_mol	kJ/mol	single row, T=298.15
S_298_J_mol_K	J/(mol K)	single row, T=298.15
cp	J/(mol K)	one row per T point
melting_point_K	K	single row
boiling_point_K	K	single row
critical_T_K	K	single row
critical_P_Pa	Pa	single row
acentric_factor	dimensionless	single row
density_kg_m3	kg/m^3	one row per (T,P) point
viscosity_Pa_s	Pa s	one row per (T,P) point
thermal_conductivity_W_mK	W/(m K)	one row per (T,P) point

CLI sub-command

```
vsepr-ufx ufx auto2 fill-thermo --batch 500 --db ufx_auto2.sqlite
vsepr-ufx ufx auto2 fill-thermo --batch 500 --nist-validate --db ufx_auto2.
    sqlite
```

`-nist-validate` fires NIST WebBook lookups for Tm, Tb, Cp(298) and writes comparison rows to `validation_records` with `validator_id='nist_thermo'`.

Continual harness integration

```
# Cycle order with Phase 7:
randomfill
fill-molecular
fill-thermo
validate        --batch $ValidateBatch
validate-web    --batch $WebBatch
promote         --min-score $MinScore
audit
```

Audit success criteria after Phase 7

- `Total property values` grows by $\geq 12\times$ `BatchSize` per cycle (12 scalar rows + curve points per record)
- `top rejected reasons` includes `thermo_block_invalid` for a non-zero fraction (confirms the check fires)
- NIST cross-check produces nonzero `web pass` and `web missing` counts
- No record reaches `validate-web` without thermo provenance rows

Phase 8 — Crystal / Material Importer (Tier 4)

Goal: attach `CrystalBlock` data to solid-phase records using Materials Project and OQMD as external references, lattice-parameter heuristics as fallback, and space-group sampling as a new generation axis.

Scope

Phase 8 applies only to records where `phase = solid`. Gas and liquid records skip `fill-crystal` without failure. The generator checks `rec.identity.phase == "solid"` before populating anything.

Axis expansion

- **space_group** — sampled from per-element-family space group frequency tables (e.g., FCC metals favour Fm-3m; hexagonal metals favour P63/mmc)
- **lattice_a_angstrom** — sampled from empirical radius-coordination correlations
- **packing_fraction** — sampled 0.52 (SC) to 0.74 (FCC/HCP)
- **energy_above_hull_eV_atom** — sampled 0.0 to 0.25 (stability filter)
- **structure_source** — one of: `generated`, `mp_lookup`, `oqmd_lookup`

Generator class: CrystalGenerator

File: `src/ufx_auto2/crystal_generator.cpp`

```
class CrystalGenerator {
public:
        CrystalGenerator();

        // Populate rec.crystal from axis sample.
```

```
        // Phase: only runs for solid-phase records.
        // Returns false (no-op) if rec.identity.phase != "solid".
        bool fill(UFXMaterialRecord& rec, const AxisSample& s) const;

        // Estimate lattice parameter a from element radius + coordination.
        static double estimate_a_angstrom(int Z,
                                          int
                                              coordination
                                              ,
                                          double
                                              ionic_radius_e
                                              )
                                              noexcept
                                              ;

        // Map coordination number + element family to most probable space
            group.
        static std::string probable_space_group(const std::string& family,
                                                int

        // Compute unit cell volume from a, b, c, alpha, beta, gamma.
        static double unit_cell_volume(const Vec3& abc,
                                       const Vec3
                                           &
                                           angles_deg
                                           )
                                           noexcept
                                           ;
private:
        // Space group frequency table: family -> vector<(sg, weight)>
        static const std::vector<std::pair<std::string,double>>&
                sg_table_(const std::string& family) noexcept;
};
\end{lstlisting>

\subsection{Materials Project fetcher}

New file: \texttt{src/ufx\_auto2/materials\_project\_fetcher.cpp}

\begin{lstlisting}
class MaterialsProjectFetcher : public IWebFetcher {
public:
        // Requires MP_API_KEY environment variable.
        // Falls back to cache-only mode if key not set.
        explicit MaterialsProjectFetcher(const std::string& api_key = "");

        WebResponse fetch(const WebQuery& q) override;

private:
        std::string api_key_;
        std::string build_url_(const WebQuery& q) const;
```

```
        WebResponse parse_response_(const std::string& body,
                                    const WebQuery& q) const;
};
```

> If `MP_API_KEY` is not set, `MaterialsProjectFetcher` operates in cache-read-only mode. It will use existing cached responses but not make new HTTP requests. This is the correct offline behavior; it is not an error.

The fetcher queries: `https://api.materialsproject.org/materials/summary/?formula={formula}&fields`

Results are cached under `web_cache/mp/{sha256_key}.json`.

LocalSanityChecker extension (Tier 4)

Applied only when `rec.identity.phase == "solid"`:

- `crystal.space_group` is non-empty
- `crystal.lattice_abc_angstrom.x > 0`
- `crystal.density_g_cm3 > 0`
- `crystal.packing_fraction` between 0.10 and 0.82
- `crystal.energy_above_hull_eV_atom` between −0.1 and 1.5 (large negative values indicate an error in the estimator)

Failure tag: `crystal_block_invalid`.

property_values rows written by fill-crystal

property_name	units	note
space_group	string	stored as value_text
space_group_number	ITA int	stored as value_real
lattice_a	Å	
lattice_b	Å	
lattice_c	Å	
lattice_alpha_deg	degrees	
unit_cell_volume	$Å^3$	
density_g_cm3	g/cm^3	at T=298.15, P=101325
packing_fraction	—	
formation_energy_eV_atom	eV/atom	
energy_above_hull_eV_atom	eV/atom	
band_gap_eV	eV	NaN if metallic
debye_temperature_K	K	estimated from elastic modulus

CLI sub-command

```
vsepr-ufx ufx auto2 fill-crystal --batch 500 --db ufx_auto2.sqlite
vsepr-ufx ufx auto2 fill-crystal --batch 500 --mp-validate --db ufx_auto2.
    sqlite
```

`-mp-validate` fires Materials Project API lookups for formation energy and space group. Writes comparison rows to `validation_records` with `validator_id='materials_project'`.

Continual harness integration

```
# Cycle order with Phase 8:
randomfill
fill-molecular
fill-thermo
fill-crystal
validate       --batch $ValidateBatch
validate-web   --batch $WebBatch
promote        --min-score $MinScore
audit
\end{lstlisting>

\subsection{Audit success criteria after Phase 8}

\begin{itemize}[leftmargin=*, itemsep=2pt]
  \item Solid-phase records have nonzero \texttt{property\_values} rows with
                \texttt{block\_name='crystal'}
  \item Gas/liquid records have zero crystal rows (phase filter working)
  \item If MP API key is set: nonzero \texttt{validator\_id='materials\
      _project'} rows
  \item \texttt{crystal\_block\_invalid} appears in top rejected reasons
\end{itemize}

% ============================================================
\section{Phase 9 --- Mechanics + Transport Macro Bridge (Tier 5)}
% ============================================================

\begin{tierbox}
\textbf{Goal:} populate \texttt{MechanicalBlock} and \texttt{TransportBlock}
    using
atomistic-to-engineering bridge correlations, and connect them to the macro
observable layer needed for pipeline erosion, thermal coupling, and flow
    modeling.
\end{tierbox}

\subsection{Context: the atomistic-to-macro chain}

Phases 2--8 built atomic-scale and molecular properties.  Phase 9 generates
the engineering-scale observables that are actually used in macro simulations
    :

\begin{center}
\texttt{UFF curvature $\rightarrow$ $E_\text{eff}$}
\quad$\cdot$\quad
\texttt{Thermo $\rightarrow$ $k_\text{eff}$}
\quad$\cdot$\quad
\texttt{Crystal density $\rightarrow$ bulk modulus estimate}
\quad$\cdot$\quad
\texttt{EOS viscosity $\rightarrow$ Darcy coefficient}
\end{center}

These are not raw DFT results.  They are engineering estimates with explicit
approximation labels.  Label them.

\subsection{Axis expansion}

\begin{itemize}[leftmargin=*, itemsep=2pt]
  \item \textbf{strain\_rate} --- sampled $10^{-6}$ to $10^3$ s$^{-1}$
  \item \textbf{test\_environment} --- one of: \texttt{vacuum}, \texttt{air},
```

```
                \texttt{molten\_salt}, \texttt{brine}, \texttt{hydrogen}
  \item \textbf{porosity} --- sampled 0.0 to 0.60
  \item \textbf{tortuosity} --- sampled 1.0 to 5.0
  \item \textbf{thermal\_gradient\_K\_m} --- sampled 0 to $10^4$
  \item \textbf{flow\_velocity\_m\_s} --- sampled 0.0 to 100.0
\end{itemize}

\subsection{Generator class: MacroPropertyGenerator}

File: \texttt{src/ufx\_auto2/macro\_property\_generator.cpp}

\begin{lstlisting}
class MacroPropertyGenerator {
public:
        MacroPropertyGenerator();

        // Fill rec.mechanical using UFF stiffness-to-modulus bridge.
        // Approximation: E ~ (r1 / D1) * coordination * k_harmonic_factor
        void fill_mechanical(UFXMaterialRecord& rec,
                                              const AxisSample& s) const;

        // Fill rec.transport using Bridgman correlations from thermo data.
        // k_eff from thermal conductivity model (kinetic theory + phonon
            scattering)
        // diffusivity from Stokes-Einstein / empirical scaling
        void fill_transport(UFXMaterialRecord& rec,
                                             const AxisSample& s) const;

        // Effective thermal conductivity via Maxwell-Garnett (porous medium)
            .
        static double k_eff_maxwell_garnett(double k_solid,
                                                                    double
                                                                        k_fluid
                                                                        ,
                                                                    double
                                                                        porosity
                                                                        )
                                                                        noexcept
                                                                        ;


        // Effective Young's modulus via Voigt bound.
        static double E_eff_voigt(double E_bulk, double porosity) noexcept;

        // Darcy permeability from Kozeny-Carman equation.
        static double permeability_kozeny_carman(double d_particle_m,
                                                                            dou









private:
        static double k_phonon_estimate_(int Z, int period,
                                                                    double
```

```
                                                                                debye_T_K
                                                                                )

                                                                                noexcept
                                                                                ;

};
\end{lstlisting>

\subsection{Bridge correlations used in Phase 9}

\begin{center}
\begin{tabular}{lll}
\toprule
\textbf{Output property} & \textbf{Input} & \textbf{Model} \\
\midrule
$E_\text{eff}$ (GPa)          & $r_1$, $D_1$, coord, density   & UFF
    stiffness bridge \\
$G_\text{eff}$ (GPa)          & $E_\text{eff}$, Poisson est.    & isotropic
    elasticity \\
$K_\text{eff}$ (GPa)          & $E_\text{eff}$, Poisson est.    & isotropic
    elasticity \\
$k_\text{eff}$ (W/m\,K)       & Debye T, density, $C_p$         & Bridgman /
    kinetic theory \\
$D$ (m$^2$/s)                 & mol.\ weight, $T$, viscosity    & Stokes-
    Einstein \\
Darcy permeability (m$^2$)    & $d_\text{particle}$, porosity   & Kozeny-
    Carman \\
\bottomrule
\end{tabular}
\end{center}

\textbf{Every output carries:}
\texttt{method\_tag = "phase9\_bridge\_heuristic"},
\texttt{confidence = 0.30}, \texttt{approximation\_label} set.

\subsection{LocalSanityChecker extension (Tier 5)}

\begin{itemize}[leftmargin=*, itemsep=2pt]
  \item \texttt{mechanical.E\_eff\_GPa} is in range $[0.001, 1200]$ GPa
  \item \texttt{transport.k\_eff\_W\_mK} is positive
  \item \texttt{transport.diffusivity\_m2\_s} is positive and below $10^{-4}$
  \item \texttt{transport.porosity} between 0.0 and 0.98 if present
  \item \texttt{mechanical.approximation\_label} non-empty
                (enforces the ``label the approximation'' hard rule)
\end{itemize}

Failure tag: \texttt{macro\_block\_invalid}.

\subsection{property\_values rows written by fill-transport and fill-
    mechanical}

\begin{center}
\begin{tabular}{lll}
\toprule
\textbf{property\_name} & \textbf{units} & \textbf{block\_name} \\
\midrule
\texttt{E\_eff\_GPa}              & GPa         & mechanical \\
\texttt{G\_eff\_GPa}              & GPa         & mechanical \\
\texttt{K\_eff\_GPa}              & GPa         & mechanical \\
\texttt{poisson\_ratio}           & \textemdash & mechanical \\
\texttt{yield\_MPa}               & MPa         & mechanical \\
\texttt{k\_eff\_W\_mK}            & W/(m K)     & transport \\
```

```
\texttt{diffusivity\_m2\_s}       & m$^2$/s     & transport \\
\texttt{permeability\_m2}         & m$^2$       & transport \\
\texttt{porosity}                 & \textemdash & transport \\
\texttt{tortuosity}               & \textemdash & transport \\
\texttt{darcy\_coeff}             & \textemdash & transport \\
\texttt{mass\_transfer\_coeff}    & m/s         & transport \\
\bottomrule
\end{tabular}
\end{center}

\subsection{CLI sub-commands}

\begin{lstlisting}[language={}]
vsepr-ufx ufx auto2 fill-transport  --batch 500 --db ufx_auto2.sqlite
vsepr-ufx ufx auto2 fill-mechanical --batch 500 --db ufx_auto2.sqlite
```

These may be called as separate commands (for partial fills) or as a combined `fill-macro` alias that calls both.

Continual harness integration

```
# Cycle order with Phase 9 (full):
randomfill
fill-molecular
fill-thermo
fill-crystal
fill-transport
fill-mechanical
validate       --batch $ValidateBatch
validate-web   --batch $WebBatch
promote        --min-score $MinScore
audit
```

Performance note: each fill step should process only records that are missing the target block rows. The query pattern is identical to the `validate` back-fill query:

```
SELECT id FROM material_records
WHERE NOT EXISTS (
  SELECT 1 FROM property_values pv
  WHERE pv.material_id = material_records.id
  AND   pv.block_name  = 'transport'
)
LIMIT ?;
```

This is idempotent. Cycles do not re-fill already-filled records.

Audit success criteria after Phase 9

- `Total property values` $\geq 30\times$ record count
- `mechanical` and `transport` block rows present for all non-rejected records
- `macro_block_invalid` in top rejected reasons for out-of-range generated estimates
- Average composite score rises above 0.85 (macro bridge contributes to S_{macro} component)

Phase 10 — Meta-Score Active Scheduler (Tier 9)

Goal: compute `MetaScoreBlock` fields for every promoted record, then use them to bias the axis sampler toward under-covered, high-usefulness, low-risk regions of parameter space. Stop generating the same octahedral iron oxide at 298 K for the thousandth time.

What Phase 10 is and is not

Phase 10 is **not** a machine learning system. It is a deterministic coverage-aware scheduler built on the `CoverageMap` that already exists in Phase 2.

Phase 10 extends `CoverageMap` with:

- per-region `coverage_score` (record count / region capacity)
- per-region `weirdness_avg` (mean weirdness of existing records in region)
- per-region `macro_usefulness_avg` (mean macro usefulness)
- per-region `extrapolation_risk` (fraction of records in region with confidence < 0.5)

Generator class: MetaScoreEngine

File: `src/ufx_auto2/meta_score_engine.cpp`

```
class MetaScoreEngine {
public:
        MetaScoreEngine();

        // Compute all MetaScoreBlock fields for a fully-filled record.
        // Requires: identity, force_field, molecular, thermo, and crystal
        // blocks populated (Phases 2-8).
        void compute(UFXMaterialRecord& rec,
                             const CoverageMap& coverage) const;

        // Bias weight for choosing the next weak axis region.
        // Higher = more likely to be sampled next.
        double region_priority(const RegionKey& region,
                                                const CoverageMap&
                                                    coverage) const
                                                    noexcept;

        // Update coverage map meta fields after a record is promoted.
        void update_coverage(CoverageMap& coverage,
                                                const UFXMaterialRecord& rec
                                                    ) const;

private:
        double weirdness_(const UFXMaterialRecord& rec,
                                        const CoverageMap& coverage) const
                                            noexcept;
        double novelty_   (const UFXMaterialRecord& rec,
                                        const CoverageMap& coverage) const
                                            noexcept;
        double macro_usefulness_(const UFXMaterialRecord& rec) const noexcept
            ;
        double extrapolation_risk_(const UFXMaterialRecord& rec) const
            noexcept;
};
\end{lstlisting>

\subsection{Composite score model}
```

```
From \texttt{UFX\_continual\_2.tex} \S19:

\begin{equation*}
S = 0.25 S_\text{identity} + 0.20 S_\text{local} + 0.20 S_\text{reference} +
    0.15 S_\text{web} + 0.10 S_\text{repeat} + 0.10 S_\text{macro}
\end{equation*}

Phase 10 populates the $S_\text{macro}$ component for the first time.
Prior phases contribute to the others.  The composite score is stored in
\texttt{property\_values} with \texttt{block\_name='meta'} and
\texttt{property\_name='composite\_score'}.

\subsection{Sampling bias: weak-region priority}

The \texttt{RandomAxisSampler} accepts an optional priority weight map.
Phase 10 injects this map at the start of each cycle via
\texttt{CoverageMap::region\_weights()}:

\begin{lstlisting}
// Priority function:
// p(region) = (1 - coverage_score)^2
//             * (1 + macro_usefulness_avg)
//             * (1 - extrapolation_risk)
//             * (1 + weirdness_avg * 0.2)
//
// Normalization: weights are renormalized to sum to 1.0 before sampling.
std::map<RegionKey, double> CoverageMap::region_weights() const;
```

This keeps the sampler deterministic (given the same seed and coverage state) while steering generation away from saturated, low-usefulness regions.

MetaScoreBlock writes to property_values

property_name	range	note
weirdness	0.0–1.0	vs. periodic neighbours
coverage	0.0–1.0	region record density
confidence	0.0–1.0	aggregate validation
novelty	0.0–1.0	distance from validated entries
local_consistency	0.0–1.0	agreement with neighbours
source_conflict	0.0–1.0	inter-source disagreement
macro_usefulness	0.0–1.0	predicts E, k, dP, damage
simulation_stability	0.0–1.0	VSEPR run stability
extrapolation_risk	0.0–1.0	model outside valid space
composite_score	0.0–1.0	promotion gate value

CLI sub-command

```
vsepr-ufx ufx auto2 score --batch 500 --db ufx_auto2.sqlite
vsepr-ufx ufx auto2 score --batch 500 --recompute --db ufx_auto2.sqlite
```

`-recompute` recalculates meta scores for all promoted records, not just those missing scores. Use after coverage map changes significantly.

Audit extensions for Phase 10

The `ufx auto2 audit` command is extended to show:

```
  -- Meta-Score Summary --
  Avg composite score   : 0.8723
  Highest weirdness     : Lu_ox3_c8_cubic_solid (0.91)
  Highest macro use     : Fe_ox2_c6_octahedral_solid (0.87)
  Highest extrap risk   : Og_ox0_c1_linear_gas (0.95)
  Weakest region        : actinide::molten_salt::octahedral
  Priority bias active  : yes  (Phase 10 scheduler)
```

Continual harness integration

```
# Cycle order with Phase 10 (full pipeline):
randomfill     --count $BatchSize
fill-molecular --batch $BatchSize
fill-thermo    --batch $BatchSize
fill-crystal   --batch $BatchSize
fill-transport --batch $BatchSize
fill-mechanical --batch $BatchSize
score          --batch $BatchSize
validate       --batch $ValidateBatch
validate-web   --batch $WebBatch
promote        --min-score $MinScore
audit
\end{lstlisting>

\begin{phasebox}
The harness parameter \texttt{-SleepSeconds} absorbs any latency from the
extended fill chain.  For the 1 GB overnight run, \texttt{-SleepSeconds 3} is
the minimum.  For the 10 GB run, use \texttt{-SleepSeconds 5} to avoid
overwhelming the web cache rate limiter.
\end{phasebox}

% ============================================================
\section{Schema Additions}
% ============================================================

Phases 6--10 add no new tables.  They use the existing generic
\texttt{property\_values} table with correct \texttt{block\_tier} and
\texttt{block\_name} values.  Dedicated tables for thermo curves, crystal
structures, or mechanical tensors may be added post-Phase 10 if query
    patterns
demand it, per the hard rule in \S\ref{sec:hardrules}.

The one schema extension is a \textbf{meta-score summary view} for audit
performance:

\begin{lstlisting}[language=SQL]
CREATE VIEW IF NOT EXISTS v_meta_scores AS
SELECT
        mr.id,
        mr.material_key,
        mr.source_class,
        MAX(CASE WHEN pv.property_name = 'composite_score'
                        THEN pv.value_real END)  AS composite_score,
        MAX(CASE WHEN pv.property_name = 'weirdness'
                        THEN pv.value_real END)  AS weirdness,
        MAX(CASE WHEN pv.property_name = 'macro_usefulness'
                        THEN pv.value_real END)  AS macro_usefulness,
        MAX(CASE WHEN pv.property_name = 'extrapolation_risk'
                        THEN pv.value_real END)  AS extrapolation_risk
FROM material_records mr
JOIN property_values pv ON pv.material_id = mr.id
```

```
WHERE pv.block_name = 'meta'
GROUP BY mr.id;
```

This view is created by `ufx_auto2_init_db()` alongside the existing six tables.

CMake Integration

Each phase adds one or two source files to the `vsepr-ufx` target in `cmake/CoreBuild.cmake`. The pattern is identical to the Phase 5 web source additions.

```
# Phase 6
src/ufx_auto2/molecular_descriptor_generator.cpp

# Phase 7
src/ufx_auto2/thermo_generator.cpp
src/ufx_auto2/nist_thermo_fetcher.cpp    # extends NISTFetcher with Cp query

# Phase 8
src/ufx_auto2/crystal_generator.cpp
src/ufx_auto2/materials_project_fetcher.cpp

# Phase 9
src/ufx_auto2/macro_property_generator.cpp

# Phase 10
src/ufx_auto2/meta_score_engine.cpp
```

No new link dependencies are needed beyond those already in the Phase 5 build (`sqlite3`, `winhttp`). The Materials Project fetcher uses the same WinHTTP path as PubChem and NIST.

File Layout Summary

File	Phase
`include/ufx_auto2/smiles_vocab.hpp`	6
`src/ufx_auto2/molecular_descriptor_generator.cpp`	6
`src/ufx_auto2/thermo_generator.cpp`	7
`src/ufx_auto2/nist_thermo_fetcher.cpp`	7
`src/ufx_auto2/crystal_generator.cpp`	8
`src/ufx_auto2/materials_project_fetcher.cpp`	8
`include/ufx_auto2/materials_project_fetcher.hpp`	8
`src/ufx_auto2/macro_property_generator.cpp`	9
`src/ufx_auto2/meta_score_engine.cpp`	10
`include/ufx_auto2/meta_score_engine.hpp`	10
`src/cli/cmd_ufx.cpp`	modified (6–10 sub-commands)
`src/cli/cmd_ufx.hpp`	modified (6–10 handler declarations)
`src/v4/uff/ufx_schema.cpp`	modified (Phase 10 view, score queries)
`cmake/CoreBuild.cmake`	modified (new sources added)
`tools/run_ufx_auto2_continual.ps1`	modified (fill steps in cycle)

Incremental Run Order

Do not implement all five phases simultaneously. The correct order:

1. **Implement Phase 6 only.** Run medium test (250 MB target). Confirm `property_values` rows appear, molecular weight cross-check fires, audit shows nonzero Tier 2 data.

2. **Implement Phase 7 only.** Run medium test again. Confirm Cp curves appear in `property_values`, NIST thermo validator produces nonzero pass/missing counts.

3. **Implement Phase 8 only.** Run 1 GB overnight candidate. Confirm solid-phase records gain crystal block, gas/liquid records do not. MP API either fires (if key set) or operates in cache-read mode.

4. **Implement Phase 9 only.** Run 1 GB overnight candidate. Confirm transport and mechanical rows appear. Average composite score should rise visibly.

5. **Implement Phase 10.** Run the 10 GB candidate. Confirm the weakest-region audit entry changes each cycle (scheduler is steering). Weirdness and macro-usefulness scores appear in audit output.

Run the tiny test (25 MB, `-EmergencyStopOnFail`) after each phase before committing to the longer runs. This is the lowest-cost proof that command wiring is correct and the harness cycle does not deadlock.

Hard Rules

1. **Fill steps process only records missing the target block.** The missing-block query is idempotent. Rerunning `fill-thermo` on a database where thermo rows already exist is a no-op. This is correct behavior and not a performance regression.

2. **Every generated value carries an approximation label.** Shomate heuristics, Bridgman correlations, UFF bridge estimates — all must have `method_tag` and `confidence` $\leq$ `0.40` unless validated by an external source.

3. **State columns are not optional.** `temperature_K`, `pressure_Pa`, and `phase` must be populated on every `property_values` row. The generation fallback is T=298.15, P=101325.0, phase=identity.phase.

4. **Generic `property_values` before dedicated tables.** Do not create `thermo_properties` or `crystal_properties` tables until the query patterns from the 1 GB run clearly require them.

5. **Phase 10 is a scheduler, not a filter.** It must not silently drop records from the cycle. It changes what gets generated next, not what gets validated or promoted.

6. **Nuclear/radiation fields never enter these fill steps.** Tier 7 (`RadiationBlock`) follows after Phase 10 is stable. The isotope/decay axis exists in `UFXMaterialRecord` and `AxisSample` but is not sampled until Tier 1 through 5 are stable.

7. **The harness cycle order is fixed.** Fill steps run before validate. Score (`fill-meta`) runs before promote. Promote never runs before validate-web on a `-WebValidate` run.

8. **SQLite is truth. Audit is the instrument panel.** If the audit output does not show the expected block counts after a fill step, the fill step is broken — not the audit.

Phase 2 proved the skeleton survives.
Phases 6–10 teach it chemistry, thermodynamics, structure, mechanics, and taste.
In that order. Not all at once.